



DNA Sequence Analysis Tool Using Advanced String-Matching Algorithms

DSA-III Mini Project

Project Report submitted for the partial fulfilment of grade for the course
Data Structures and Algorithms – 3 (DSA-III) | A.Y. 2026-2027

Submitted by

S. Yuva Mahesh (2520030336) • K. Shloka (2520030334) • Sreshta (2520030217)

Team Number: 6 | Section: 5

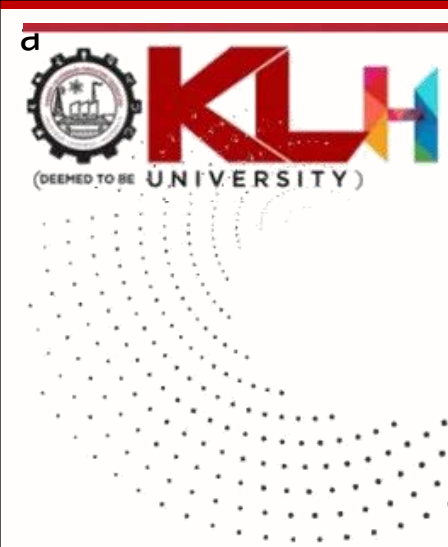
Under the guidance of

MS D SWATHI

Assistant Professor, CSE

Contents

1. Abstract
2. Introduction
3. Literature Survey
4. Problem Statement and Objectives
5. Flow Chart
6. Results
7. Hardware and Software Requirements
8. Conclusion
9. Future Scope



Abstract

- Java-based web application (DSA-III mini-project) for DNA motif searching and performance analysis.
- Implements four string-matching algorithms: KMP, Z-Algorithm, Rabin-Karp, and Aho-Corasick.
- KMP, Z-Algorithm and Rabin-Karp search one motif at a time; Aho-Corasick searches many motifs simultaneously.
- Processes DNA sequences built from four nucleotide bases: A, T, G, C.
- Users paste a sequence or upload a .txt/.fasta file, then search for one or more motifs.
- Identifies match positions, highlights matches, times each algorithm, and reports the fastest one.
- Runs entirely in memory - no database required.

Introduction

- DNA is a sequence built from just four letters: A (Adenine), T (Thymine), G (Guanine), C (Cytosine).
- A recurring short sequence within DNA - like a start codon or a promoter region - is called a motif.
- Finding every occurrence of a motif inside a long genome is a classic string-matching problem.
- A naive character-by-character search takes $O(n \cdot m)$ time - too slow for large genome files.
- Advanced algorithms (KMP, Z-Algorithm, Rabin-Karp, Aho-Corasick) solve this in linear or near-linear time.
- This project builds a practical tool that applies all four algorithms to real DNA-style data and compares them head-to-head.

Literature Survey

Algorithm	Origin	Core Idea
Knuth-Morris-Pratt (KMP)	Knuth, Morris & Pratt (1977)	Failure function (LPS array) avoids re-checking matched characters.
Z-Algorithm	Gusfield, Algorithms on Strings, Trees and Sequences (1997)	Z-array gives the longest match with the prefix at every position.
Rabin-Karp	Rabin & Karp (1987)	Polynomial rolling hash compares windows in $O(1)$ before a direct check.
Aho-Corasick	Aho & Corasick (1975)	Trie of all patterns plus KMP-style failure links; finds every pattern in one pass.



Problem Statement and Objectives

Problem Statement :

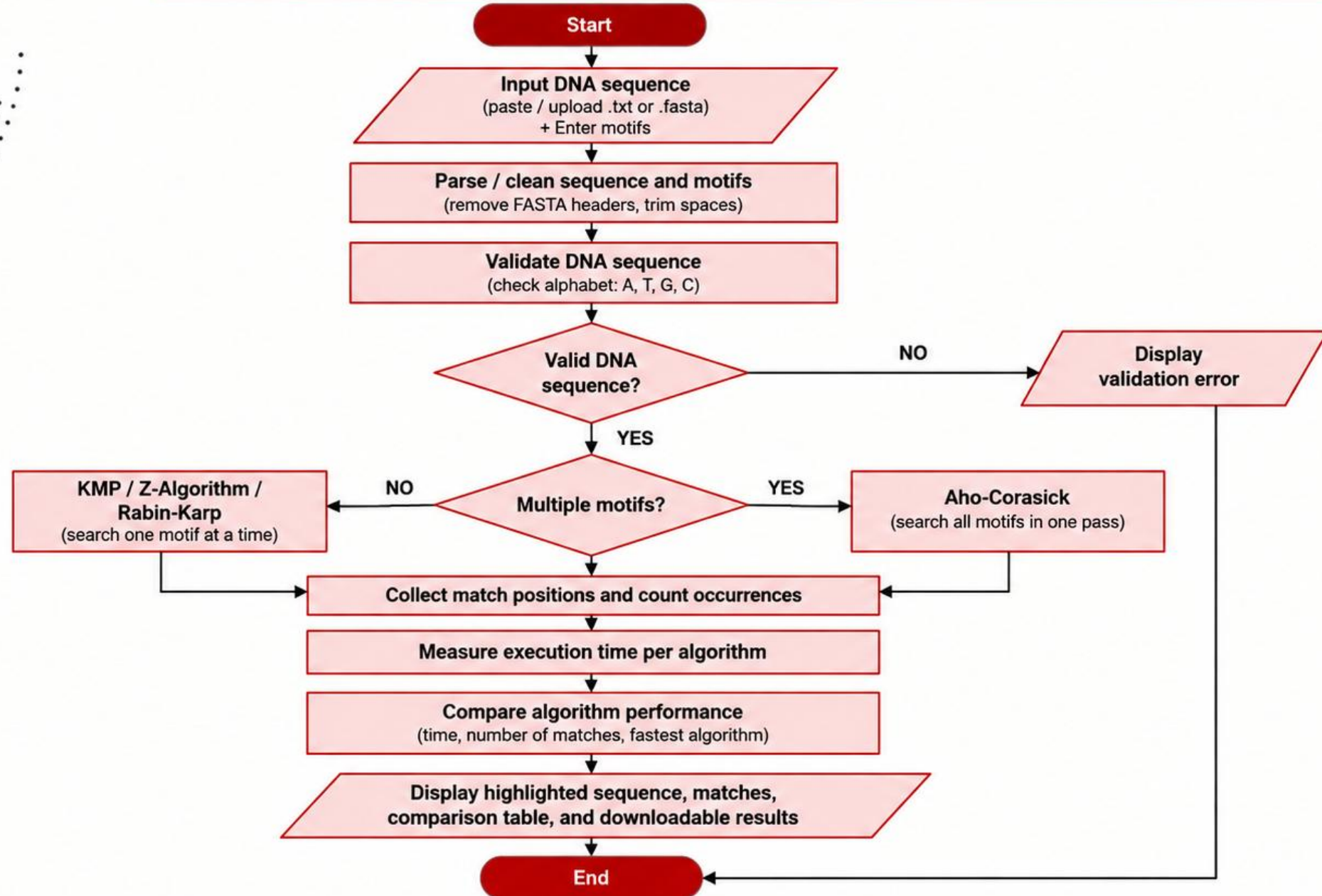
Searching for specific DNA motifs within large genome sequences using **naive character-by-character comparison** can require **$O(n \cdot m)$** time for each search, making repeated searches computationally expensive as the sequence size grows. This project addresses this challenge by applying **advanced string-matching algorithms** to efficiently perform single-pattern and multi-pattern searches, accurately identify motif locations, and compare algorithm performance on DNA sequence data.

Objectives :

The main objective of the project is to **accurately locate DNA motif patterns** within sequence data using advanced string-matching techniques. The system also aims to **support simultaneous searching of multiple motifs** using the Aho-Corasick algorithm, while **measuring and comparing actual execution times** with the theoretical time and space complexity of each algorithm. It validates DNA input according to the nucleotide alphabet **{A, T, G, C}** and is designed to operate **entirely offline without a database or external services**.



Flow Chart



Results

Sample run: sequence length 118 bases, motifs = {ATG, TAGC, GTAC, ATGC}

Algorithm	Occurrences	Time (ms)	Time Complexity
KMP	26	2.5341	$O(n + m)$
Z-Algorithm	26	1.2021	$O(n + m)$
Rabin-Karp ★ Fastest	26	0.6217	$O(n + m)$ avg
Aho-Corasick	26	2.5820	$O(n + m + z)$

- All four algorithms agree exactly on the number and position of every match, confirming correctness.
- Rabin-Karp was fastest in this run; results vary by input size and motif count.
- The application highlights every match directly within the sequence and lets the user download the full report.

Hardware and Software requirements

Hardware

- Any standard desktop or laptop (Windows / macOS / Linux)
- Minimum 4 GB RAM recommended
- No GPU or special hardware required

Software

- Java Development Kit (JDK) 17 or later
- Any modern web browser (Chrome, Edge, Firefox)
- No Maven, no Spring Boot, no database, no internet required to run

Conclusion

- The **DNA Sequence Analysis Tool** successfully demonstrates the practical application of four advanced string-matching algorithms—**KMP, Z-Algorithm, Rabin-Karp, and Aho-Corasick**—for accurate DNA motif searching. KMP, Z-Algorithm, and Rabin-Karp efficiently handle single-motif searches, while Aho-Corasick supports multiple motifs in a single traversal of the sequence. The system provides accurate match locations and performance measurements, enabling meaningful comparison of the implemented algorithms. It processes DNA-style data using the nucleotide bases **A, T, G, and C** and operates locally without requiring a database or external services, making it a lightweight and effective platform for demonstrating advanced DSA concepts in **bioinformatics and genome sequence analysis**.

Future Scope

- The future scope of the project includes extending the tool with **fuzzy motif matching** using Edit Distance and Smith-Waterman alignment, **reverse-complement searching** for both DNA strands, and **suffix array with LCP-based analysis** for identifying similar or repeated regions across genome files. The system can also be enhanced through **parallel processing of chromosome-sized sequences** to improve performance on very large datasets, along with **persistent search history** for cross-session analysis. Additional improvements could include **interactive visualizations** for motif distribution and algorithm performance, better memory optimization, and support for genome datasets of **100 MB or larger**, making the tool more scalable and useful for practical bioinformatics applications.



INPUT & OUTPUT

Explorer

Open Editors

DnaAnalyzer\$AhoCorasick\$Node.class

DnaAnalyzer\$AlgoStat.class

DnaAnalyzer\$DnaUtil.class

DnaAnalyzer\$HistoryEntry.class

DnaAnalyzer\$HistoryHandler.class

DnaAnalyzer\$KMP.class

DnaAnalyzer\$Match.class

DnaAnalyzer\$MultipartData.class

DnaAnalyzer\$OutputStreamWrapper.class

▼ dna-analyzer-simple-updated

sample-data

sample_genome.fasta

DnaAnalyzer.class

DnaAnalyzer.java

DnaAnalyzer\$AboutHandler.class

DnaAnalyzer\$AhoCorasick.class

DnaAnalyzer\$AhoCorasick\$Node.class

DnaAnalyzer\$AlgoStat.class

DnaAnalyzer\$DnaUtil.class

DnaAnalyzer\$HistoryEntry.class

DnaAnalyzer\$HistoryHandler.class

DnaAnalyzer\$KMP.class

DnaAnalyzer\$Match.class

DnaAnalyzer\$MultipartData.class

DnaAnalyzer\$OutputStreamWrapper.class

DnaAnalyzer\$RabinKarp.class

DnaAnalyzer\$RootHandler.class

DnaAnalyzer\$SearchHandler.class

DnaAnalyzer\$SearchResult.class

DnaAnalyzer\$ZAlgo.class

① README.md

> Outline

> Timeline

> Projects

> Run Configuration

> Java Projects

8 import java.io.OutputStream;

9 import java.net.InetSocketAddress;

10 import java.net.URLDecoder;

11 import java.nio.charset.StandardCharsets;

12 import java.time.LocalDateTime;

13 import java.time.format.DateTimeFormatter;

14 import java.util.ArrayDeque;

15 import java.util.ArrayList;

16 import java.util.Arrays;

17 import java.util.Collections;

18 import java.util.Comparator;

19 import java.util.HashMap;

20 import java.util.LinkedHashMap;

21 import java.util.List;

22 import java.util.Locale;

23 import java.util.Map;

24 import java.util.concurrent.Executor;

25 import java.util.concurrent.atomic.AtomicInteger;

26 import java.util.regex.Matcher;

27 import java.util.regex.Pattern;

28

29 public class DnaAnalyzer {

30 private static final int PORT = 8080;

31 private static final List<HistoryEntry> HISTORY = Collections.synchronizedList(new ArrayList());

32 private static final int MAX_HISTORY = 20;

33 private static final AtomicInteger HISTORY_ID_SEQ = new AtomicInteger(1);

34 private static final Pattern HEADER_PARAM_PATTERN_TEMPLATE = Pattern.compile("");

35 private static final String CSS = ":root {\n --bg: #eef3f6; --bg-accent1: #e4f3ef; --bg-accent2: #e4f3ef; }";

36

37 public DnaAnalyzer() {

Problems

Output

Debug Console

Terminal

Ports

Spell Checker

JavaSE-25 LTS

Active code page: 65001

C:\Users\yuvam\OneDrive\Documents\Year 02\DSA-3\PRJ\dna-analyzer-simple-updated>javac DnaAnalyzer.java

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\yuvam\OneDrive\Documents\Year 02\DSA-3\PRJ\dna-analyzer-simple-updated>java DnaAnalyzer

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

=====

DNA Sequence Analysis Tool started successfully

Open: http://localhost:8080/

Stop: press Ctrl+C in this terminal

=====

DNA Sequence Analyzer

Home History About Toggle Dark/Light

DNA Sequence Analysis Tool

Search DNA motifs using KMP, Z-Algorithm, Rabin-Karp and Aho-Corasick.

DNA Sequence (paste it, or choose a file below)

>ENST00000435737.5 cds chromosome:GRCh38:3:112039620:112081269:1 gene:ENSG00000176040.13
gene_biotype:protein_coding transcript_biotype:nonsense_mediated_decay gene_symbol:TMPRSS7
description:transmembrane protease, serine 7 [Source:HGNC Symbol;Acc:HGNC:30846]
ATGTTTCGCATCACCAACATTGAGTTTCTCCGAAATACCGACAAAAGGAGTCCAGGGAA
TTTCTTTCAGTGTACGGACTGTGCAGCAAGTGATAAACCCTGGTTTATACACATCTGCC
TTCTCCAAATTTATGAGCAGTCTGTGTTGTCAGATGTGAGCAACAAAGGCGGCTC
CTTGTCACCTTTTGATTGTTTTTGTGTCATGCCACGTGCCAAGGCCACATCTTCTGTGAA
GACTGTGTTGCCGCCATCTTGAAGGACTCCATCCAGACAAAGCATATAACCGGACCTCT
GTGGGGAGCTTGCCAGGACTGGCTGTGGACATGGACTCTGTGCTACTAATGAAGTCTCTG

Or upload a .txt / .fasta file

Choose file No file chosen

Motifs to search (one per line, or comma separated)

ATG
GCC
TATA

Run Search (All 4 Algorithms)

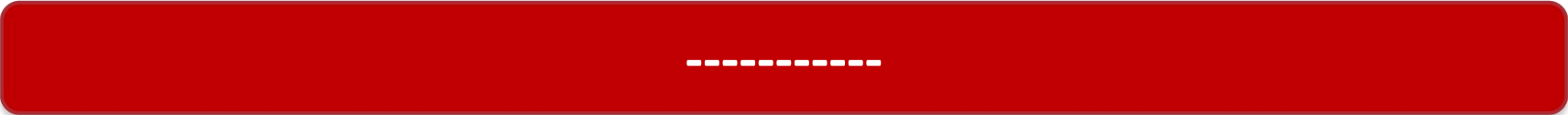
Results Dashboard

2544
SEQUENCE LENGTH

3
MOTIFS SEARCHED

99
TOTAL OCCURRENCES

Rabin-Karp
BEST ALGORITHM



Thank you

DNA Sequence Analysis Tool Using Advanced String-Matching Algorithms | Team 6